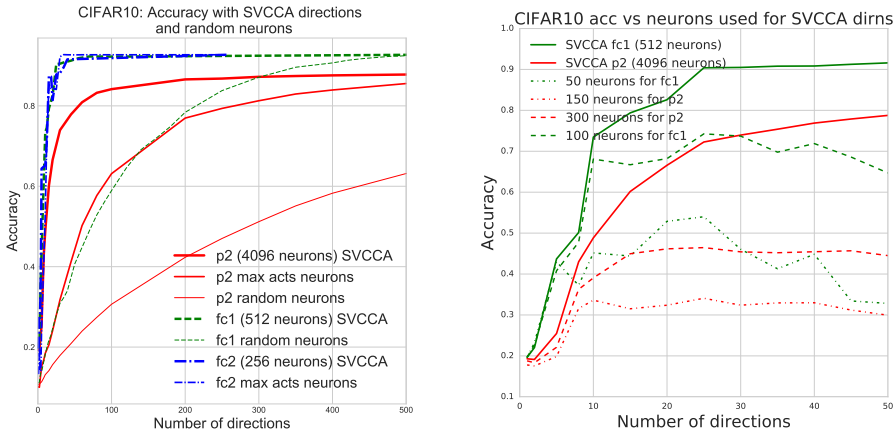

(a) (b)

Figure 2: Demonstration of *(a)* disproportionate importance of SVCCA directions, and *(b)* distributed nature of some of these directions. For both panes, we first find the top k SVCCA directions by training two conv nets on CIFAR-10 and comparing corresponding layers. *(a)* We project the output of the top three layers, pool1, fc1, fc2, onto this top-k subspace. We see accuracy rises rapidly with increasing k, with even $k \ll \text{num neurons}$ giving reasonable performance, with *no* retraining. Baselines of random k neuron subspaces and max activation neurons require larger k to perform as well. *(b)*: after projecting onto top k subspace (like left), dotted lines then project again onto m neurons, chosen to correspond highly to the top k-SVCCA subspace. Many more neurons are needed than k for better performance, suggesting distributedness of SVCCA directions.

$\{\tilde{\boldsymbol{z}}_1^{l_1}, ..., \tilde{\boldsymbol{z}}_m^{l_1}\}$ and $\{\tilde{\boldsymbol{z}}_1^{l_2}, ..., \tilde{\boldsymbol{z}}_m^{l_2}\}$, ordered according to importance by SVCCA, with each $\tilde{\boldsymbol{z}}_j^{l_i}$ being a linear combination of the original neurons, i.e. $\tilde{\boldsymbol{z}}_j^{l_i} = \sum_{r=1}^{m} \alpha_{jr}^{(l_i)} \boldsymbol{z}_r^{l_i}$.

For different values of $k < m$, we can then restrict layer l_i's output to lie in the subspace of $\text{span}(\tilde{\boldsymbol{z}}_1^{l_i}, \ldots, \tilde{\boldsymbol{z}}_k^{l_i})$, the most useful k-dimensional subspace as found by SVCCA, done by projecting each neuron into this k dimensional space.

We find — somewhat surprisingly — that very few SVCCA directions are required for the network to perform the task well. As shown in Figure 2(a), for a network trained on CIFAR-10, the first 25 dimensions provide nearly the same accuracy as using all 512 dimensions of a fully connected layer with 512 neurons. The accuracy curve rises rapidly with the first few SVCCA directions, and plateaus quickly afterwards, for $k \ll m$. This suggests that the useful information contained in m neurons is well summarized by the subspace formed by the top k SVCCA directions. Two baselines for comparison are picking random and maximum activation neuron aligned subspaces and projecting outputs onto these. Both of these baselines require far more directions (in this case: neurons) before matching the accuracy achieved by the SVCCA directions. These results also suggest approaches to model compression, which are explored in more detail in Section 4.4.

Figure 2(b) next demonstrates that these useful SVCCA directions are at least somewhat distributed over neurons rather than axis-aligned. First, the top k SVCCA directions are picked and the representation is projected onto this subspace. Next, the representation is further projected onto m neurons, where the m are chosen as those most important to the SVCCA directions . The resulting accuracy is plotted for different choices of k (given by x-axis) and different choices of m (different lines). That, for example, keeping even 100 fc1 neurons (dashed green line) cannot maintain the accuracy of the first 20 SVCCA directions (solid green line at x-axis 20) suggests that those 20 SVCCA directions are distributed across 5 or more neurons each, on average. Figure 3 shows a further demonstration of the effect on the output of projecting onto top SVCCA directions, here for the toy regression case.

Why the two step SV + CCA method is needed. Both SVD and CCA have important properties for analysing network representations and SVCCA consequently benefits greatly from being a two step method. CCA is **invariant** to affine transformations, enabling comparisons without natural alignment (e.g. different architectures, Section 4.4). See Appendix B for proofs and a demonstrative figure. While CCA is a powerful method, it also suffers from certain shortcomings, particularly in determining how many directions were important to the original space X, which is the strength of

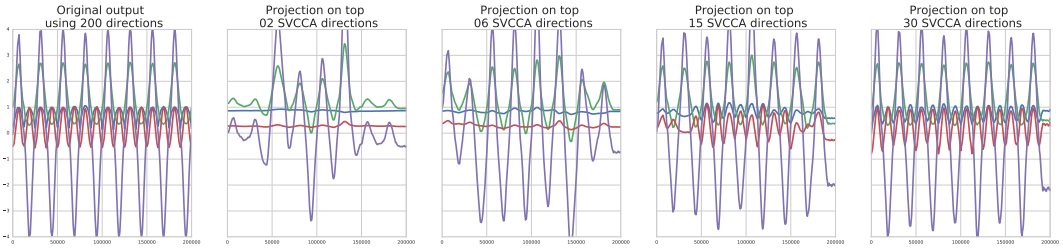

Figure 3: The effect on the output of a latent representation being projected onto top SVCCA directions in the toy regression task. Representations of the penultimate layer are projected onto $2, 6, 15, 30$ top SVCCA directions (from second pane). By 30, the output looks very similar to the full 200 neuron output (left).

SVD. See Appendix for an example where naive CCA performs badly. Both the SVD and CCA steps are critical to the analysis of learning dynamics in Section 4.1.

3 Scaling SVCCA for Convolutional Layers

Applying SVCCA to convolutional layers can be done in two natural ways:

(1) *Same layer comparisons:* If X, Y are the same layer (at different timesteps or across random initializations) receiving the same input we can concatenate along the pixel (height h, width w) coordinates to form a vector: a conv layer $h \times w \times c$ maps to c vectors, each of dimension hwd, where d is the number of datapoints. This is a natural choice because neurons at different pixel coordinates see *different* image data patches to each other. When X, Y are two versions of the same layer, these c different views correspond perfectly.

(2) *Different layer comparisons:* When X, Y are not the same layer, the image patches seen by different neurons have no natural correspondence. But we can flatten an $h \times w \times c$ conv into hwc neurons, each of dimension d. This approach is valid for convs in different networks or at different depths.

3.1 Scaling SVCCA with Discrete Fourier Transforms

Applying SVCCA to convolutions introduces a computational challenge: the number of neurons ($h \times w \times c$) in convolutional layers, especially early ones, is very large, making SVCCA prohibitively expensive due to the large matrices involved. Luckily the problem of approximate dimensionality reduction of large matrices is well studied, and efficient algorithms exist, e.g. [4].

For convolutional layers however, we can avoid dimensionality reduction and perform *exact* SVCCA, even for large networks. This is achieved by preprocessing each channel with a Discrete Fourier Transform (which preserves CCA due to invariances, see Appendix), causing all (covariance) matrices to be block-diagonal. This allows all matrix operations to be performed block by block, and only over the diagonal blocks, vastly reducing computation. We show:

Theorem 1. *Suppose we have a translation invariant (image) dataset X and convolutional layers l_1, l_2. Letting $DFT(l_i)$ denote the discrete fourier transform applied to each channel of l_i, the covariance $cov(DFT(l_1), DFT(l_2))$ is block diagonal, with blocks of size $c \times c$.*

We make only two assumptions: 1) all layers below l_1, l_2 are either conv or pooling layers with circular boundary conditions (translation equivariance) 2) The dataset X has all translations of the images X_i. This is necessary in the proof for certain symmetries in neuron activations, but these symmetries typically exist in natural images even without translation invariance, as shown in Figure App.2 in the Appendix. Below are key statements, with proofs in Appendix.

Definition 1. *Say a single channel image dataset X of images is translation invariant if for any (wlog $n \times n$) image $X_i \in X$, with pixel values $\{\boldsymbol{z}_{11}, ...\boldsymbol{z}_{nn}\}$, $X_i^{(a,b)} = \{\boldsymbol{z}_{\sigma_a(1)\sigma_b(1)}, ...\boldsymbol{z}_{\sigma_a(n)\sigma_b(n)}\}$ is also in X, for all $0 \leq a, b \leq n-1$, where $\sigma_a(i) = a + i \mod n$ (and similarly for b).*

For a multiple channel image X_i, an (a, b) translation is an (a, b) height/width shift on every channel separately. X is then translation invariant as above.

To prove Theorem 1, we first show another theorem:

Theorem 2. *Given a translation invariant dataset X, and a convolutional layer l with channels $\{c_1, \dots c_k\}$ applied to X*

(a) the DFT of c_i, FcF^T has diagonal covariance matrix (with itself).

(b) the DFT of c_i, c_j, Fc_iF^T, Fc_jF^T have diagonal covariance with each other.

Finally, both of these theorems rely on properties of *circulant matrices* and their DFTs:

Lemma 1. *The covariance matrix of c_i applied to translation invariant X is circulant and block circulant.*

Lemma 2. *The DFT of a circulant matrix is diagonal.*

4 Applications of SVCCA

4.1 Learning Dynamics with SVCCA

We can use SVCCA as a window into learning dynamics by comparing the representation at a layer at different points during training to its final representation. Furthermore, as the SVCCA computations are relatively cheap to compute compared to methods that require training an auxiliary network for each comparison [1, 10, 11], we can compare all layers during training at all timesteps to all layers at the final time step, producing a rich view into the learning process.

The outputs of SVCCA are the aligned directions $(\tilde{x}_i, \tilde{y}_i)$, how well they align, ρ_i, as well as intermediate output from the first step, of singular values and directions, $\lambda_X^{(i)}, x'^{(i)}$, $\lambda_Y^{(j)}, y'^{(j)}$. We condense these outputs into a single value, the *SVCCA similarity* $\bar{\rho}$, that encapsulates how well the representations of two layers are aligned with each other,

$$\bar{\rho} = \frac{1}{\min(m_1, m_2)} \sum_i \rho_i, \tag{1}$$

where $\min(m_1, m_2)$ is the size of the smaller of the two layers being compared. The SVCCA similarity $\bar{\rho}$ is the average correlation across aligned directions, and is a direct multidimensional analogue of Pearson correlation.

The SVCCA similarity for all pairs of layers, and all time steps, is shown in Figure 4 for a convnet and a resnet architecture trained on CIFAR10.

4.2 Freeze Training

Observing in Figure 4 that networks broadly converge from the bottom up, we propose a training method where we successively *freeze* lower layers during training, only updating higher and higher layers, saving *all* computation needed for deriving gradients and updating in lower layers.

We apply this method to convolutional and residual networks trained on CIFAR-10, Figure 5, using a linear freezing regime: in the convolutional network, each layer is frozen at a fraction (layer number/total layers) of total training time, while for resnets, each residual block is frozen at a fraction (block number/total blocks). The vertical grey dotted lines show which steps have another set of layers frozen. Aside from saving computation, Freeze Training appears to actively *help* generalization accuracy, like early stopping but with different layers requiring different stopping points.

4.3 Interpreting Representations: when are classes learned?

We also can use SVCCA to compare how correlated representations in each layer are with the logits of each class in order to measure how knowledge about the target evolves throughout the network. In Figure 6 we apply the DFT CCA technique on the Imagenet Resnet [6]. We take five different classes and for different layers in the network, compute the DFT CCA similarity between the logit of that class and the network layer. The results successfully reflect semantic aspects of the classes: the firetruck class sensitivity line is clearly distinct from the two pairs of dog breeds, and network develops greater sensitivity to firetruck earlier on. The two pairs of dog breeds, purposefully chosen so that each pair is similar to the other in appearance, have cca similarity lines that are very close to each other through the network, indicating these classes are similar to each other.